

Enterprise Programming (Course Slides)

Module5

Refer for detailed notes :-

<https://github.com/sagaruppuluri/EP/blob/main/Module5/Readme.md>

Messaging, Security & Deployment

A Comprehensive Guide for Modern Java Enterprise Development



JMS



JWT Security



Logging



Caching



Docker & CI



Git

Agenda

01 Messaging with JMS

Asynchronous communication using Point-to-Point and Pub/Sub patterns with ActiveMQ

02 REST API Security

Securing APIs with JWT (JSON Web Token) authentication in Spring Boot 3

03 Logging Frameworks

Structured logging with Logback and SLF4J for monitoring and debugging

04 Caching with Redis

Performance optimization using Spring Boot's `@Cacheable` with Redis backend

05 Docker & CI/CD

Containerization with Docker and automated pipelines using Jenkins

06 Git & GitHub

Version control fundamentals, branching strategies, and collaboration workflows



Messaging with JMS

Asynchronous communication between Java components

Why Messaging?



Asynchronous

Producer and consumer do not need to be active simultaneously



Decoupled

Components evolve independently — no direct dependencies



Scalable

Add more consumers to handle higher load without code changes



Reliable

Messages persist in the broker even if the receiver is temporarily down

JMS & ActiveMQ

JMS (Java Message Service)

Standard Java API for creating, sending, receiving, and reading messages between distributed systems.

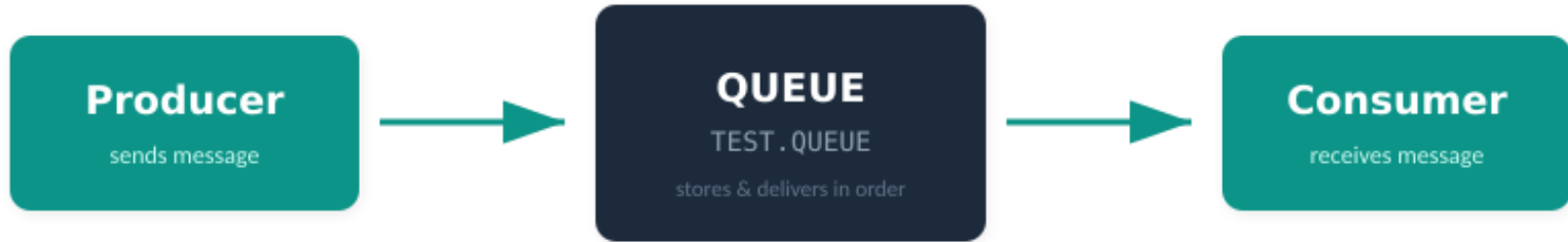
ActiveMQ

Popular open-source message broker by Apache. Runs on `tcp://localhost:61616`. Supports both Queues and Topics.

Two Core Patterns:

- P2P (Queue) — one-to-one delivery
- Pub/Sub (Topic) — one-to-many broadcast

Point-to-Point (P2P) Pattern



One message is consumed by exactly one consumer

One Consumer

Each message is consumed by exactly one receiver from the Queue

Ordered Delivery

Messages are delivered in FIFO order for reliable processing

Use Cases

Order processing, task distribution, request-reply workflows

Publish / Subscribe (Pub/Sub) Pattern



Pub/Sub Uses Topics

One message delivered to ALL active subscribers.
Great for notifications, event feeds, and real-time updates.

P2P vs Pub/Sub

P2P → Queue → one consumer gets each message
Pub/Sub → Topic → all subscribers get every message

JMS Producer – Sending Messages to a Queue

JMSProducer.java

```
// 1. Create connection factory pointing to ActiveMQ broker
ConnectionFactory factory = new ActiveMQConnectionFactory("tcp://localhost:61616");

// 2. Establish connection and create session
Connection connection = factory.createConnection();
connection.start();
Session session = connection.createSession  false, Session.AUTO_ACKNOWLEDGE);

// 3. Create the destination queue
Destination destination = session.createQueue("TEST.QUEUE");

// 4. Create producer and send a text message
MessageProducer producer = session.createProducer(destination);
producer.setDeliveryMode(DeliveryMode.NON_PERSISTENT);
TextMessage message = session.createTextMessage("Hello, JMS!");
producer.send(message);

// 5. Always close resources in reverse order
producer.close(); session.close(); connection.close();
```

JMS Consumer — Receiving Messages from a Queue

JMSConsumer.java

```
// Same connection setup as producer (factory, connection, session)
// Create consumer bound to same queue name
Destination destination = session.createQueue("TEST.QUEUE");
MessageConsumer consumer = session.createConsumer(destination);

// Poll for messages with a timeout loop
while (true) {
    Message message = consumer.receive(1000); // 1-second timeout

    if (message instanceof TextMessage) {
        TextMessage textMsg = (TextMessage) message;
        System.out.println("Received: " + textMsg.getText());
    } else {
        System.out.println("Received non-text message");
    }
}

consumer.close(); session.close(); connection.close();

// Maven: org.apache.activemq:activemq-all:5.16.3
```

Pub/Sub Code – Publisher & Subscriber

JMSPublisher.java

```
// Key change: createTopic instead
// of createQueue
Destination dest =
    session.createTopic("TEST.TOPIC");

MessageProducer producer =
    Session.createProducer(dest);
producer.setDeliveryMode(
    DeliveryMode.NON_PERSISTENT);

TextMessage msg =
    session.createTextMessage(
        "Hello, JMS Pub/Sub!");

producer.send(msg);
```

JMSSubscriber.java

```
// Subscribe to same topic name
Destination dest =
    session.createTopic("TEST.TOPIC");

MessageConsumer consumer =
    session.createConsumer(dest);

// Receive loop (same as P2P)
while (true) {
    Message msg = consumer.receive(1000);
    if (msg == null) break;
    if (msg instanceof TextMessage) {
        // Process message
    }
}
```

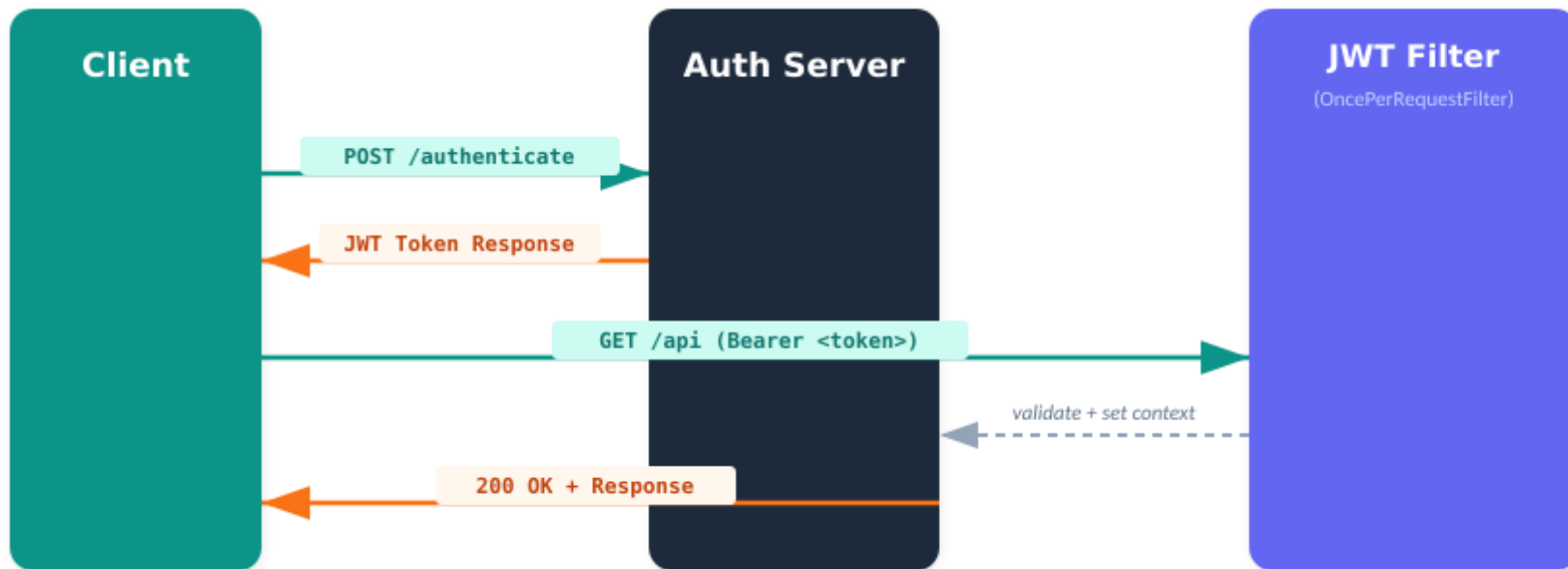
 **TIP: Run multiple JMSSubscriber instances simultaneously to observe broadcast behavior.**



REST API Security

JWT Authentication with Spring Boot 3

JWT Authentication Flow



JWT (JSON Web Token) is a self-contained, stateless token. The server doesn't store session data — the token itself carries the user's identity and claims, signed with a secret key to prevent tampering.

JWT Implementation – Key Components

JwtUtil.java

- generateToken(username)
→ signs JWT with HS256
- extractUsername(token)
→ reads subject claim
- validateToken() → checks
expiry + identity match
- Secret key + 10-hour
expiration window

JwtRequestFilter.java

- Extends
OncePerRequestFilter
- Extracts Bearer token from
Authorization header
- Calls JwtUtil to validate
the token
- Sets SecurityContext if
token is valid

SecurityConfig.java

- Disables CSRF (stateless
REST API)
- /authenticate endpoint is
publicly accessible
- All other endpoints
require valid JWT
- SessionCreationPolicy.STATELESS enforced

Testing the JWT Authentication Flow

Get JWT Token

1

```
curl -X POST http://localhost:8080/authenticate \
  -H "Content-Type: application/json" \
  -d '{"username":"user1","password":"password1"}'
```

Access Protected Endpoint

2

```
curl -X GET http://localhost:8080/hello \
  -H "Authorization: Bearer <YOUR_JWT_TOKEN>"
```

Expected Results

3

Valid token	→	200 OK + "Hello, authenticated user!"
Expired token	→	401 Unauthorized
No token	→	403 Forbidden



Logging Frameworks

Structured logging for monitoring, debugging, and auditing

Java Logging Ecosystem

Log4j

The original Java logging framework. Highly configurable with appenders for console, file, database, and more. Supports multiple log levels.

SLF4J

A logging facade (abstraction layer). Write code against SLF4J API, then swap backends (Logback, Log4j) at deploy time — zero code changes.

Logback

Modern successor to Log4j with better performance. Auto-reloads configuration, native SLF4J integration. Default in Spring Boot.

Log Levels — Severity Order (Low to High)

TRACE

Finest detail

DEBUG

Diagnostic info

INFO

Key events

WARN

Potential issues

ERROR

Failures

Logback – Configuration & Usage

src/main/resources/logback.xml

```
<configuration>

  <appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>
        %d{yyyy-MM-dd HH:mm:ss}%-5level
        %logger{36} - %msg%n
      </pattern>
    </encoder>
  </appender>

  <root level="debug">
    <appender-ref ref="CONSOLE" />
  </root>
</configuration>
```

LoggingController.java

```
@RestController
public class LoggingController {

    Logger logger =
LoggerFactory.getLogger(
    LoggingController.class);

    @GetMapping("/log")
    public String logExample() {
        logger.debug("DEBUG message");
        logger.info("INFO message");
        logger.warn("WARN message");
        logger.error("ERROR message");
        return "Check console logs!";
    }
}

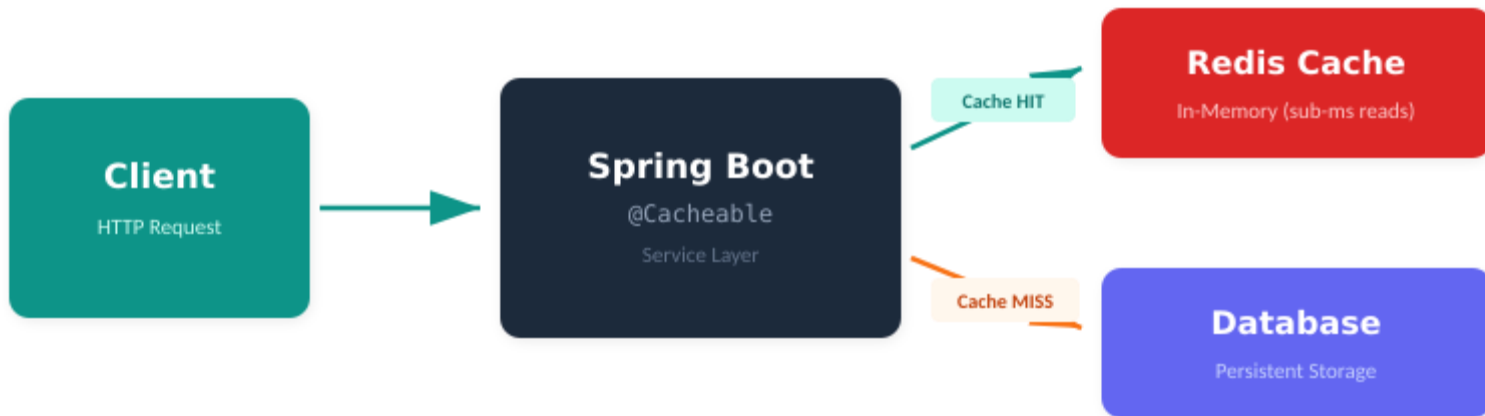
// Test: curl localhost:8080/log
```



Caching with Redis

Dramatically improve response times with in-memory data caching

How Redis Caching Works in Spring Boot



Cache HIT

Data found in Redis → returned in < 1ms. No database call needed.

Cache MISS

Data not in cache → fetch from DB → store result in Redis for next time.

Why Redis?

In-memory speed, rich data structures (strings, hashes, lists, sets), high scalability.

Spring Boot + Redis: @Cacheable in Action

CachingService.java

```
@Service
public class CachingService {

    @Autowired
    private RedisTemplate<String,Object>
        redisTemplate;

    // @Cacheable stores the return
    // value in Redis on first call.
    // Subsequent calls with same id
    // return cached result instantly.

    @Cacheable(value = "dataCache",
               key = "#id")
    public String getDataById(String id)
    {
        Thread.sleep(3000); // Slow DB
        return "Data for ID: " + id;
    }
}
// 1st call: ~3 sec | 2nd+: < 1 ms
```

Configuration + Controller

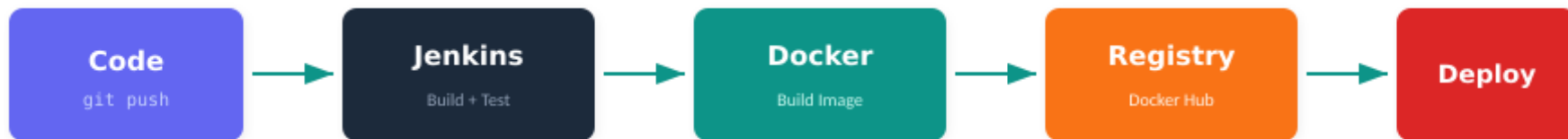
```
# application.properties
spring.redis.host=localhost
spring.redis.port=6379
---
@Configuration @EnableCaching
public class CacheConfig {
    @Bean
    public CacheManager cacheManager(
        RedisTemplate<String,Object> rt) {
        return RedisCacheManager.builder(
            rt.getConnectionFactory())
            .build();
    }
}
---
@RestController
public class CachingController {
    @Autowired CachingService svc;
    @GetMapping("/cached-data/{id}")
    public String get(@PathVariable id){
        return svc.getDataById(id);
    }
}
```



Docker & CI/CD

Containerization and automated deployment pipelines

CI/CD Pipeline – From Code to Deployment



Continuous Integration / Continuous Deployment Pipeline

Docker – Containerization

- Packages app + all dependencies into a single portable container
- Uses host OS kernel — far lighter than traditional VMs
- Consistent behavior across dev, test, and production
- Build once, run anywhere with docker run

Jenkins – CI/CD Automation

- Open-source automation server for build pipelines
- Triggers automatically on git push events
- Pipeline-as-code via Jenkinsfile in your repo
- Stages: Build → Test → Docker → Push → Deploy

Dockerfile & Essential Docker Commands

Dockerfile

```
# Base image with JDK 17
FROM openjdk:17-jdk-slim

# Set container working directory
WORKDIR /app

# Copy the built JAR into container
COPY target/my-app.jar app.jar

# Expose port for Spring Boot
EXPOSE 8080

# Start the application
ENTRYPOINT ["java", "-jar", "app.jar"]

# ----- Build & Run -----
# docker build -t my-app .
# docker run -p 8080:8080 my-app
```

Docker Cheat Sheet

<code>docker run redis</code>	<code># run image</code>
<code>docker images</code>	<code># list images</code>
<code>docker ps</code>	<code># running only</code>
<code>docker ps -a</code>	<code># all containers</code>
<code>docker stop <id></code>	<code># stop container</code>
<code>docker rm <id></code>	<code># remove container</code>
<code>docker rmi <image></code>	<code># remove image</code>
<code>docker logs <id></code>	<code># view logs</code>
<code>docker exec -it <id> bash</code>	<code># shell</code>
<code># Push to Docker Hub</code>	
<code>docker tag my-app user/my-app</code>	
<code>docker push user/my-app:latest</code>	

Jenkinsfile – Pipeline as Code

Jenkinsfile (Declarative Pipeline)

```
pipeline {
  agent any
  stages {
    stage('Build')          { steps { sh 'mvn clean package -DskipTests' } }
    stage('Docker Build') { steps { sh 'docker build -t my-app .' } }
    stage('Push Registry'){ steps {
      sh'docker tag my-app user/my-app:latest'
      sh'docker push user/my-app:latest'
    }}
    stage('Deploy')         { steps { sh 'kubectl apply -f deployment.yaml' } }
  }
}
```

Jenkins Setup Steps

1. Install Jenkins + plugins
(Git, Docker)

2. Create Pipeline job

3. Point to Git repo SCM

4. Specify Jenkinsfile path

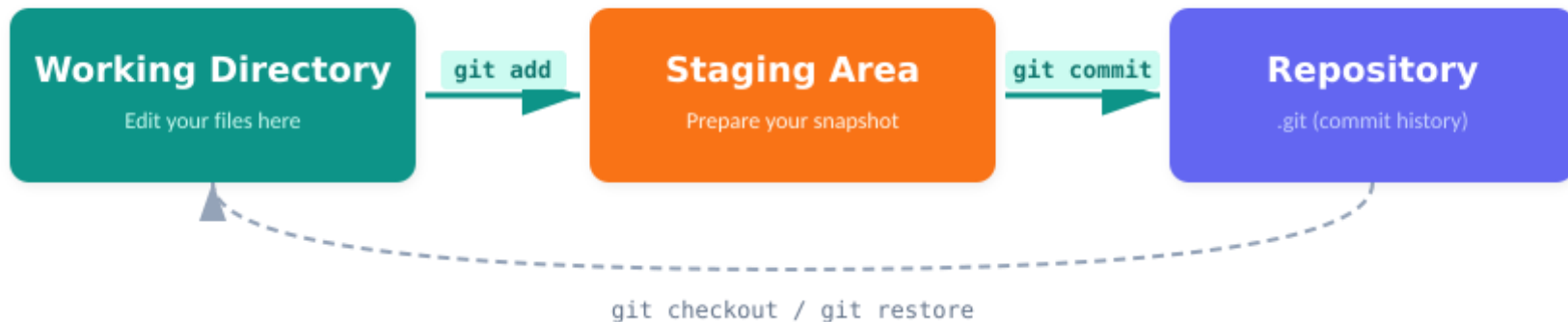
5. Auto-triggers on push



Git & GitHub

Version control, branching, and team collaboration

Git's Three-Tree Architecture



Git Supports Every Phase of Development

Development

Create branches for features, make local commits to track changes iteratively.

Testing

Isolate experiments in branches. Merge only after tests pass on the feature branch.

Deployment

Tag releases, use hooks to trigger CI/CD pipelines automatically on push.

Essential Git Commands

Command	Description
<code>git init</code>	Initialize a new repository in current directory
<code>git remote add origin <url></code>	Link local repo to remote GitHub repository
<code>git add .</code>	Stage all modified and new files for commit
<code>git commit -m "message"</code>	Record staged changes with a descriptive message
<code>git checkout -b <branch></code>	Create and switch to a new feature branch
<code>git push origin <branch></code>	Upload local commits to the remote repository
<code>git pull origin main</code>	Fetch and merge latest remote changes into local
<code>git merge <branch></code>	Combine another branch into current branch
<code>git log --oneline</code>	View compact one-line commit history
<code>git status</code>	Show current working tree and staging status

Setting Up a GitHub Repository

1

Log in to GitHub and navigate to the Repositories tab

2

Click "New" to create a new repository

3

Enter repository name, optional description, and choose visibility (public or private)

4

Click "Create repository" to finalize setup

5

Copy the repository URL and connect your local project using `git remote add origin <url>`

Key Takeaways



JMS Messaging

P2P (Queue) for one-to-one, Pub/Sub (Topic) for broadcast. ActiveMQ as the broker.



JWT Security

Stateless authentication: generate token at login, validate on every API request.



Logging

Logback + SLF4J for structured logging. Configure levels per environment.



Redis Caching

@Cacheable for instant repeat reads. Massive performance gains with minimal code.



Docker + CI/CD

Containerize with Dockerfile. Automate build-test-deploy with Jenkinsfile.



Git & GitHub

Branch, commit, push, collaborate. Version control is the foundation of teamwork.



Thank You!

Questions & Discussion

Practice hands-on with ActiveMQ, Docker, and Git.
Experiment, break things, and build your understanding through doing.